How to add a custom voxel-based dynamic model into CASToR

April 2, 2019

Foreword for developers

Before adding some code to CASToR, it is highly recommended to read the general documentation *CASToR__general_documentation.pdf* to get a good picture of the project, as well as the programming guidelines *CASToR_HowTo__programming_guidelines.pdf*. Also, the philosophy about adding new modules in CASToR (*e.g.* projectors, optimizers, deformations, dynamic models, etc) is fully explained in *CASToR_HowTo__add_new_modules.pdf*. Finally, the doxygen documentation is a very good resource to help understanding the code architecture.

Still TODO in the document:
- figures
- castor-ImageBasedDynamicModel
- revoir section 7
- Description of NestedEM (iLinearModel) and NNLS (vDynamicModel)

1 Summary

This HowTo guide describes how to add a new dynamic model class for applications such as temporal regularization, or kinetic modeling to derive parametric images. The model could be applied between the images of chronological frames of a dynamic acquisition (simply refered by *frames* in the CASToR nomenclature and in this document) or between the images of respiratory and/or cardiac gates of a gated dataset (whose events belonging to the same state (i.e phase or amplitude) of a physiological motion have been previously regrouped into several *gates*). CASToR is mainly designed to be modular in the sense that adding a new feature should be easy. This guide begins with a general description of the dynamic model part of the CASToR architecture that explains the chosen implementation. Then follows one step-by-step guide that explains how to add a new dynamic model by simply adding a new class with few mandatory requirements.

2 Command-line options related to dynamic reconstruction

CASToR implements different command line options in order to manage a dynamic datafile. These options are presented below. Note that information about these command-line options presented below can also be displayed from the code executable using the option *-help-dynamic*.

2.1 Framing (-frm)

This option allows to split the dataset in time frames whose duration is defined by the user. When enabled, a different image will be reconstructed for each time frame. The attribution of each data element (lines of response for list-mode, or histogram bin) to a specific frame is performed using the timestamp tag of each events. These events are assumed to be chronologically ordered in the dataset (both for list-mode and histogram. In the latter case, the 3D histogram datasets belonging to each frame are supposed to be chronologically concatenated), except if gating is involved (see next paragraph below).

The time splitting of the data could be provided to the main code with the command line option -frm list, where 'list' is a list of all frames durations in seconds separated by commas. To specify a dead frame, add 'df' concatenated to the frame duration. The maximum precision is milliseconds. (default: 1 frame of the whole input file duration)".

2.2 Gating (-g)

A dynamic dataset can be separated into subsets of data elements (list-mode lines of response, histogram bins) belonging to a specific part of a physiological motion (respiratory, cardiac, or a combination of both). In this case, the events are expected to be ordered by gate (i.e the events belong to the same state (i.e phase or amplitude) of a physiological motion), and not follow a chronological ordering. For a 5D datafile incorporating both dynamic frames and gates, the gates belonging to each frames are supposed to be concatenated.

Several information regarding the gating of the data have to be provided in the reconstruction command-line options using the **-g** option along with a text file containing gating information, in order to enable gating management. The required information are the number of respiratory/cardiac gates, and the number of data elements (events) inside each gates. These informations could be given either in the datafile header, or from a separate text file whose path after the -g command-line option (see section 7 for more details).

Figure 1 presents an example of a gating set up file for a dynamic dataset containing 10 frames and 8 (respiratory) gates. The gating associated to each frame must be entered on a separate line. The number of events in each gate are separated by commas. (TODO : Once dual-gating is validated, rather show a file with double gating in order to illustrate with the most complex case). Using the **-g** command-line option and this set up, an image will be reconstructed for each gate within each frame.

```
# -- RESPIRATORY GATING --

nb_respiratory_gates            : 8

respiratory_gates_data_splitting            :
1516563,1500341,1743174,1779358,1686703,1642721,1646636,1626992
1466996,1456050,1686886,1722672,1630495,1590838,1591864,1577053
1428700,1417418,1643586,1675922,1590997,1549552,1550657,1536055
1397140,1383691,1604895,1639988,1552764,1512370,1511894,1499491
1374240,1363131,1576879,1605845,1526048,1486712,1488037,1472384
1340110,1326465,1539962,1567528,1490828,1450041,1453035,1434547
1306997,1291690,1496126,1530189,1448835,1411017,1414313,1397439
1280463,1265994,1470635,1501725,1422078,1383648,1386512,1372475
1250712,1232485,1433921,1468570,1390024,1350492,1352661,1340273
2438649,2409871,2794120,2858313,2707544,2631801,2636037,2612423


# -- CARDIAC GATING --

nb_cardiac_gates            : 1

cardiac_gates_data_splitting            :
```

Figure 1: Set up of a gating file related to a dynamic dataset with respiratory gating

3 Dynamic model initialization

A dynamic model module can be enabled in order to perform temporal based processing, such as kinetic modelling or temporal regularization, between the images of each frame (or gate).

The **-dynamic-model** command line option allows to specify a dynamic model and its potential configuration parameters (see details in section 7). CASToR currently contains a *LinearModel* class, and a *Patlak* class dedicated to kinetic modelling for irreversible radiotracers. Both class are described in the following sections. More details about the available dynamic models and information regarding their initialization can be displayed using the command **-help-dmodel**.

3.1 iLinearModel:

This class implements a general linear dynamic model applied between the images of a dynamic acquisition. The model is applied on a voxel-by-voxel basis between either the images of the frames or respiratory / cardiac gates. Several models could be used simultaneously on 5D/6D datasets (i.e dynamic gated acquisitions splitted in chronological time frames, or dual-gated acquisitions). At each iteration and subset, the parametric images of the model, and optionally the basis functions of the model, are estimated using the nested EM algorithm. Several variables control different parameters of the algorithm, such as the number of iterations of EM iterations for the model, the ratio of parametric images updates before basis function updates. This class must be initialized using a configuration file with the following command-line option:

```
-dynamic-model LinearModel:/path/to/conf/file.txt
```

The configuration file must contain one (or several) couple(s) of BEGIN/END keywords to define at which dynamic level of modelling the user-provided parameters must be applied:

- DYNAMIC FRAMING / ENDDF : model related to the chronological dynamic frames of the dynamic dataset
- RESPIRATORY GATING / ENDRG : model related to the respiratory gates of the dynamic dataset
- CARDIAC GATING / ENDCG : model related to the cardiac gates of the dynamic dataset

The following parameters could be defined inside these keywords to set a dynamic model at a specific dynamic level:

- **Number basis functions: x** (mandatory)

  The number x of basis functions (and parametric images) defined in the model

- **Basis functions: list** (mandatory)

  Basis function initial values (list of coefficients for each frame or gate, separated by commas. Each functions must be entered on new lines)

- **Parametric images init: path** (optional)

  Parametric images initialization using an interfile image located at the provided *path*. Without initialization, all voxel will be set to 1 by default)

The following parameters are common to all level of dynamic models:

- **Number model iterations: x**

  Number of iterations of the model parameters and basis functions updates in one cycle of Nested EM (one cycle consists in x iterations in which either the parametric images or the basis functions are updated) (Default x == 1)

- **Basis function update ratio: x**

  Ratio of update between parametric images and basis functions updates cycle. Cycles consist in x iterations of the parametric images, following by x iterations of the basis functions. If x == 0, only the parametric images are updated (Default x == 0).

- **Basis function start ite : x**

  Starting iteration for the update of basis functions. If negative, no update of the basis functions is performed (only parametric images are updated) (Default x == -1).

Listing 1: Example of LinearModel initialization.

```
1
2   DYNAMIC FRAMING
3   Number basis functions          : 2
4   Basis_functions             :
5   23682.79, 25228.74, 26636.99, 27923.61, 29101.16
6   5.4, 4.91, 4.48, 4.1, 3.75
7   ENDDF
8
9   RESPIRATORY GATING
10  Number basis functions          : 6
11  Basis_functions             :
12  1, 0.8, 0.6, 0.4, 0.2, 0.01
13  0.7, 0.9, 0.7, 0.5, 0.3, 0.1
14  0.4, 0.6, 0.8, 0.6, 0.4, 0.2
15  0.2, 0.4, 0.6, 0.8, 0.6, 0.4
16  0.1, 0.3, 0.5, 0.alpin7, 0.9, 0.7
17  0.01, 0.2, 0.4, 0.6, 0.8, 1
18  ENDRG
19
20  Number_model_iterations : 1
21  Basis_function_start_ite : -1
22  Basis_function_update_ratio : 0
```

3.2 iLinearPatlakModel:

This class implements the Patlak Reference Tissue Model (Patlak CS, Blasberg RG: Graphical evaluation of blood-to-brain transfer constants from multiple-time uptake data. J Cereb Blood Flow Metab 1985, 5(4):5 84-590. DOI http://dx.doi.org/10.1038/jcbfm.1985.87)

It is used to model radiotracers which follows as 2-tissue compartment model with irreversible trapping. The Patlak temporal basis functions are composed of the Patlak slope (integral of the reference TAC from the injection time divided by the instantaneous reference activity), and intercept (reference tissue TAC).

It can be initialized using either a configuration file or a list of option with the following keywords and information:

3.2.1 configuration file:

The file must be provided to the *castor-recon* main executable using the following command-line options.

```
-dynamic-model Patlak:/path/to/conf/file.txt
```

As this class inherits from the *LinearModel* class. The parameters must be declared inside a couple of the following specific tags:

- DYNAMIC FRAMING/ENDDF

The following parameters must be defined inside these keywords to set a dynamic model at a specific dynamic level:

- **Basis functions: list** (mandatory)

  The coefficients of Patlak plot and intercept for each time frame (tf), on two successive lines, separated by ',' : Basis functions: coeffPplottf1, coeffPplottf2, ..., coeffPplottfn coeffPintctf1, coeffPintctf2, ..., coeffPintctfn

- **Parametric images init: path** (optional)

  Parametric images initialization using an interfile image located at the provided *path*. Without initialization, all voxel will be set to 1.0 and 1000. for the Patlak slope and intercept by default)

- **Patlak method: x** (optional) optimization method for Patlak parameters estimation: x=0: Least-Square (default) x=1: Iterative non-negative Least-Square (C.L. Lawson and R.J. Hanson, Solving Least Squares Problems) x=2: Nested EM (todo)

3.2.2 Command line options:

The Patlak class initialization can be directly made from the command line options. Thelist of options must contain the coefficients of both Patlak functions separated by commas, with the following template :

```
-dynamic-model Patlak,coeffPplottf1,coeffPplottf2,...,coeffPplottfn,
               coeffPintctf1,coeffPintctf2,...,coeffPintctfn
```

The parametric images will be initialized with 1.0 and 1000. for Patlak slope and intercept by default. The parametric images estimations will be written on disk for each iteration. Default optimization method is least-square.

3.3 Parameters common to all dynamic model:

The following keywords are optional and common to each dynamic model (Dynamic frame, Respiratory gating or Cardiac gating). They control the main functions dedicated to the estimation of the model parameters, and the re-estimation of the image using the model:

- **Number of iterations before image update: x**

  Set a number x of iteration to reach before using the model to generate the images at each frames/gates (Default x == 0)

- **No image update: x**

  If x set to 1, the reconstructed images for the next iteration/subset are not reestimated using the model (the code just performs standard independent reconstruction of each frames/gates) (Default x = 0).

- **No parameters update: x**

  If set to 1, the parameters / functions of the model are not estimated with the image (this could be used to test The EstimateImageWithModel() function with specific user-provided parametric images) (Default x = 0).

- **Save parametric images : x**

  Enable (1)/Disable(0) saving parametric images on disk (Default x == 1)

4 The dynamic module architecture

The main classes managing dynamic models are *oDynamicModelManager* and *vDynamicModel*, located in the dynamic subfolder. The main program will instantiate and initialize the *oDynamicModelManager* and the dynamic model class inheriting from *vDynamicModel* if enabled. During the iterative reconstruction process, at the end of each iteration/subset loop, the *ApplyDynamicModel()* function of the manager will call the *EstimateModel()* and *EstimateImage()* functions. The first will successively call the *EstimateModelParameters()* and *EstimateImageWithModel()* functions of the child class. As their names suggest, the first function is dedicated to the estimation of any parameters of the model, while the second function is used to regenerate an image from the model. The addition of a new dynamic model mostly require to implement these two functions. Note that these functions can be individually enabled/disabled depending on the values of some parameters implemented by *vDynamicModel* specific to vDynamicModel as specified in section 3.3. Please have a look to the next section 5 for more help about the dynamic model implementation itself.

5 Add your own dynamic model

5.1 Basic concept

The addition of a new image-based dynamic model require to build a specific class that inherits from the abstract class *vDynamicModel*. Then, one just has to implement a set of pure virtual functions for the initialization and application of the model. Please refer to the *CASToR_HowTo__add_new_modules.pdf* guide in order to fill up the mandatory parts of adding a new module; namely the auto-inclusion mechanism, the interface-related functions and the management functions. To ease the implementation, a template class is provided in the source code and already implements all the squeleton. Basically, one will have to change the name of the class and fill the related functions up in his own code. The actual files are *include/dynamic/iDynamicModelTemplate.hh* and *src/dynamic/iDynamicModelTemplate.cc* and are actually already part of the source code. Right below are some instructions to help fill the specific pure virtual projection functions of a dynamic model.

5.2 Implementation of the dynamic model functions

Several mandatory functions should be implemented (or return an error by default) in a new dynamic model class :

ShowHelp(): Simply output some help and guidance to describe what the dynamic model does and how to use it.

ReadAndCheckOptionsList(): Implement here the reading of any options specific to this dynamic model passed through the argument `const string& a_listOptions`. The user can make use of the *ReadStringOption()* function to parse the list of parameters. If the function is not mandatory (e.g initialization of the model using a file is required rather than with command-line parameters), just send an error message and return 1.

ReadAndCheckConfigurationFile(): Implement here the reading of any configuration file specific to this dynamic model, passed through the argument `const string& a_fileOptions`. The user can make use of the *ReadDataASCIIFile()* function to read data from a file. If the function is not mandatory (e.g initialization of the model using command line options is required rather than with a file), just send an error message and return 1.

CheckParameters(): Use this function to check if the private parameters of your class have correctly been initialized. It might be a good idea to initialize all private parameters in the constructor with default erroneous value to properly check their initialization.

InitializeSpecific(): Use this function to Instanciate/Initialize any member variables/arrays after the parameters have been checked in the previous function.

EstimateModelParameters(): The estimation of the model parameters and parametric images must be implemented here, with the help of any private functions if required. The *ap_ImageS* parameter allows to access the image matrices in the oImageSpace class. The current estimation of the image *m4p_image* can be accessed from there. It contains 4 dimensions in order to access to any dynamic level :

$ap\_ImageS- > m4p\_image[fr][rg][cg][v]$
fr = time frames
rg = respiratory gates
cg = cardiac gates
v = actual voxel of the 3D volume

EstimateImageWithModel(): This function can be used to generate the serie of dynamic images from the model (i.e, the image matrix $ap\_ImageS- > m4p\_image$), after estimation of the model parameters in EstimateModelParameters(). It is called right after EstimateModelParameters()

All information and the tools needed to implement these functions are fully described in the template source file *src/dynamic/iDynamicModelTemplate.cc*, so please refer to it.

6 castor-ImageBasedDynamicModel toolkit

// TODO

7 Meta-data command line options

List of the different set of options related to image-based dynamic models :

- **-g** *file*: Give a text file defining the dynamic data splitting (due to respiratory/cardiac gating or involuntary patient motion correction). The file should contain the some of the following keywords :

  `nb_respiratory_gates`: number of respiratory gates in the data

  `respiratory_gates_data_splitting`: enter the number of events within each gate, separated by commas. If the data contains several frame (dynamic acquisition), the data splitting of each frame should be entered on a new line

  `nb_cardiac_gates`: number of cardiac gates in the data

  `cardiac_gates_data_splitting`: enter the number of events within each gate, separated by commas. If the data contains several frame (dynamic acquisition), the data splitting of each frame should be entered on a new line.

  `nb_involuntary_motion_triggers`: number of involuntary patient motion triggers in the data

  `listt_involuntary_motion_triggers`: enter the timestamp of each transformation, separated by commas. No specific nomenclature is required for dynamic acquisitions (data containing several frames). However, if the data contains several bed positions, the data splitting of each bed should be entered on a new line.

- **-rm** *param*: Give the deformation to be used for respiratory motion correction, along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).

- **-cm** *param*: Give the deformation to be used for cardiac motion correction, along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).

- **-rcm** *param*: Give the deformation to be used for dual gating motion correction (both respiratory and cardiac), along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).

- **-im** *param*: Give the deformation to be used for patient motion correction, along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).